01 — Harness and Evaluation

This is the technical contract. It is based primarily on the competition's `HARNESS_README.md` (S5), with the starter notebook (S6) and host/community reports where noted. Line references are exact for the local copies in this workspace.

The short version: **you submit declarative YAML; the harness compiles it into an ADK agent, serves one fixed Gemma 4 model on 4×L4, runs the agent in a sandbox, extracts a git diff, applies it to a fresh sandbox with hidden tests, and scores pass/fail.** There is no arbitrary Python entrypoint, and test-runner edits are stripped/reset rather than honored.

---

1. Stack and security model

The evaluation stack has three libraries (S5:21-23):

Library	Role
`adk-submission`	Declarative agent compiler and inference-server manager. Compiles untrusted submission directories into ADK agent trees without `importlib`.
`adk-eval-core`	Task/result models, Docker/subprocess sandboxes, resilient file editing, token/cost tracking, ADK plugins, ATIF v1.7 trajectory tracing.
`swegemma`	SWE-bench-style lifecycle: `Evaluator`, `run_agent_sandbox`, `verify_task`, the nine tools, prompt/nudges, diff extraction, pytest+JUnit scoring.

The security posture is **declarative-only** (S5:57):

- No `agent.py` or `agent_fn` is executed.
- All agents, sub-agents, workflows, tools, and generation parameters are declared in YAML.
- Tools resolve against closed registries (`ToolRegistry`, `ModelRegistry`, `SkillRegistry`, `CallbackRegistry`).
- The submission cannot import arbitrary Python.
- S5 permits `.py` for ADK skill scripts (S5:130) and the skill toolset exposes `run_skill_script` (S5:326), but S5 does not state where the script executes. Treat skill scripts as untrusted code with unknown host/network reach until verified; prefer prompts and YAML over scripts.

This is why "write a custom Python tool" is not an option. The available tools are the nine built-ins (plus skill tools/sub-agents). Design within that constraint. S5 permits `.py` for ADK skill scripts but does not specify the execution boundary; treat skill scripts as untrusted.

---

2. Submission contract

2.1 Archive layout

The canonical tree is (S5:63-82; the `scripts/`/`resources/` subdirectories are shown in the competition overview, S1):

```
submission.zip
├── agent.yaml                  # required root config
├── eval_config.yaml            # optional budgets
├── configs/
│   └── sampling.yaml           # optional generation params via !include
├── prompts/
│   ├── system.md               # optional instruction via !include
│   └── analyzer.md
├── sub_agents/
│   └── code_analyzer.yaml      # optional sub-agent / AgentTool
├── adapters/
│   └── main_lora/
│       ├── adapter_config.json
│       └── adapter_model.safetensors
└── skills/
    └── repo_navigation/
        ├── SKILL.md
        ├── scripts/
        └── resources/
```

Root config discovery: exactly one of `agent.yaml`, `agent.yml`, `root_agent.yaml`, `root_agent.yml` must exist at the submission root (S5:84). Zero → `MissingRootConfigError`; more than one → `MultipleRootConfigsError`.

Packaging ambiguity: the README says the directory is packaged "at the root of `submission.zip`" (S5:61) but draws a top-level `submission/` node (S5:63). The starter notebook uses `root_dir=AGENT_DIR`, which puts `agent.yaml` at the archive root (S6:600-601). Use that: `cd agent_dir && zip -r ../submission.zip .` so `agent.yaml` is the first path. Validate with `unzip -l`.

Dummy adapters: the provided `sample_submission` ships `adapters/main_lora` and `adapters/tool_lora` dummy adapters, and `discover_adapters()` registers every adapter (S5:191-201, S6:246-247). Delete `adapters/` and remove the `adapter:` references unless you trained one; otherwise you pay extra LoRA memory/KV-cache cost for untrained weights and may point at a missing adapter.

Extraction target: S5:166 mentions `/kaggle/working/submission`; the exact scorer path is not otherwise specified. Package the zip with `agent.yaml` at the root.

Participants upload `submission.zip`; the scorer runs `scripts/inference.py` and writes `[id, prediction]` to `submission.parquet` (S5:61,537). You do not produce the parquet yourself, and its schema does not constrain the agent's output beyond the patch string.

2.2 `eval_config.yaml`

Optional, at root, under `evaluation:` (S5:85, S5:493-497). The notebook also accepts top-level keys as a fallback (`raw_eval_cfg.get("evaluation", raw_eval_cfg)`, S6:341), but use `evaluation:` for clarity. The notebook reads `verification_timeout_seconds` (S6:344); the scorer fixes Phase-2 at 300 s (S5:506).

```
evaluation:
  timeout_seconds: 300
  max_tool_calls: 10
  max_time_minutes: 1
  max_turns: 50
```

Important: `evaluation.timeout_seconds` maps to the **per-command** timeout (`config.harness.command_timeout_seconds`), not session wall clock (S5:500,505). Session wall clock is `evaluation.max_time_minutes`. The starter values (1 minute / 10 calls / 50 turns) are a smoke test, not a performance baseline; the starter run observed 0/2 `NO_PATCH` (S6:491-516).

2.3 `!include`

`!include <relative_path>` (S5:86-89):

- `.md` / `.txt` → raw UTF-8 string (for `instruction` / `global_instruction`).
- `.yaml` / `.yml` → parsed recursively, max depth 10, cycle detection.
- Blocked: absolute paths, null bytes, `..` traversal, symlinks outside the submission root → `PathTraversalError`.

Use this for prompts and sampling configs; keep paths relative.

2.4 Hard limits

Constraint	Value	Source
Unpacked submission size	README says **<3 GiB** and gives 3,221,225,472 bytes (exactly 3 GiB); the notebook asserts `<=`. Stay strictly below 3 GiB, including adapters.	S5:115,120; S6:606-607
Files	10,000	S5:121
YAML files	1,000	S5:122
YAML size	50 MiB per file / cumulative include expansion	S5:123
Skill size	50 MiB per skill directory	S5:124
Instruction chars	1,000,000 per agent; 10,000,000 total	S5:125
Agents	500	S5:126

Constraint	Value	Source
Sub-agent depth	50	S5:127
Skills	1,000	S5:128
LoopAgent iterations	1–500, default 500	S5:112,129

Allowed extensions (S5:130): `.yaml`, `.yml`, `.md`, `.txt`, `.py` (skill scripts), `.json` (`adapter_config.json`), `.safetensors` (`adapter_model.safetensors`). Pickle weights (`.bin`, `.pt`, `.pth`) and other binary/archive extensions are rejected. Adapters must be `.safetensors` only (S5:131).

Ambiguities to avoid: don't put loose `.py` files outside `skills/`; don't rely on a top-level folder in the zip; don't exceed 3 GiB unpacked even if the zip is smaller. These are safety choices, not exploits.

---

3. Agent schema

3.1 Agent classes

Four discriminated types; `agent_class` omitted defaults to `LlmAgent` (S5:92):

- **`LlmAgent`** — the normal model-driven agent.
- **`SequentialAgent`** — runs `sub_agents` in order (S5:108).
- **`ParallelAgent`** — runs `sub_agents` concurrently in isolated branches (S5:110).
- **`LoopAgent`** — repeats `sub_agents` up to `max_iterations` (default 500, bounded 1–500) (S5:112).

3.2 `LlmAgent` fields

Field	Meaning	Notes
`name`	Agent identifier	Required (S5:95)
`model`	Model alias	Required unless a default is set; use an allowed model (S5:96)
`adapter`	Adapter name under `adapters/`	Optional; per-agent (S5:97)
`description`	Used by parent when delegating	Up to 1M chars (S5:98)
`instruction`	System prompt	Supports `{problem_description}`,

Field	Meaning	Notes
		`{hints}`, `output_key` variables (S5:99)
`tools`	Tool names, `agent_tool` refs, inline agent dicts	See below (S5:100)
`skills`	Relative skill directory paths	Optional (S5:101)
`sub_agents`	`{config_path: ...}` references	Transfer-based delegation (S5:102)
`output_key`	Saves final text to `session.state[output_key]`	Optional (S5:103)
`include_contents`	`"default"` or `"none"`	Whether history is passed (S5:104)
`disallow_transfer_to_parent` / `..._to_peers`	Booleans	Semantics not fully documented (S5:105)
`generate_content_config`	Sampling/thinking config	See below (S5:106)

3.3 Tools

`tools` accepts (S5:100):

- built-in tool names: `run_command`, `submit_patch`, `get_status`, `read_file`, `edit_file`, `write_file`, `get_code_neighbors`, `search_similar_code`, `get_code_subgraph`;
- `agent_tool: {config_path: sub_agents/foo.yaml, skip_summarization: true}`;
- inline agent dictionaries.

The harness filters the initial prompt's tool-specific guidance to the tools actually declared across the agent hierarchy (S5:282). If you don't declare graph tools, the model won't be told about them. If you declare them but don't want them used, say so in the prompt (the public prompt-only config, author-reported 0.24, does exactly that; S36).

3.4 `generate_content_config` and thinking

Allowed fields (S5:132-137):

- `temperature`, `top_p`, `top_k`, `max_output_tokens`, `presence_penalty`, `frequency_penalty`, `stop_sequences`, `response_mime_type`, `seed`, `thinking_config`.
- `max_output_tokens`: 1–32,768; default 16,384; bounded by vLLM `max_model_len=32768`.
- `thinking_config.thinking_budget`: 0–32,768; default 4,096; `>0` enables thinking and forwards `extra_body.thinking_token_budget`; `0` disables.

- `thinking_config.thinking_level`: `MINIMAL`/`LOW`/`MEDIUM`/`HIGH`/`NONE`; on vLLM `/v1` (especially with LoRA), **omit it** and use `include_thoughts: true` + `thinking_budget`.
- `thinking_config.include_thoughts`: `true` keeps thought parts in the response and later tool-call context; `false` still enables reasoning on the current turn but strips thoughts from the response/history; if neither budget nor level is set, it acts as the fallback `enable_thinking` toggle.
- Rejected inside `generate_content_config`: `tools`, `system_instruction`, `http_options`, `safety_settings`, `response_schema` (S5:139).

The public prompt-only config (author-reported 0.24) uses `max_output_tokens: 8192`, `thinking_budget: 4096`, `include_thoughts: true`, `temperature: 0.2`, `top_p: 0.95` (S36). This is the public reference; the primary 5-min bundle uses thinking 2048. `max_output_tokens` above 8192 can make vLLM reject long prompts because prompt + requested output must fit 32,768 (S38).

---

4. Models, serving, and LoRA

4.1 Hardware and vLLM

The scorer runs on **4×NVIDIA L4, 24 GB each, 96 GB total** (S5:144-146). vLLM is configured with (S5:147-154):

- `tensor_parallel_size = 4`
- `gpu_memory_utilization = 0.90` (~21.6 GB/GPU, ~86.4 GB total)
- `max_model_len = 32768` (prompt + reasoning + output combined)
- `scheduler_reserve_full_isl = False`
- `enable_auto_tool_choice = True`, `tool_call_parser = "gemma4"`, `reasoning_parser = "gemma4"`, reasoning start/end `<|channel>`/`<channel|>`
- LoRA right-sizing: `enable_lora = bool(adapter_map)`, `max_loras = max(1, len(adapter_map))`, `max_lora_rank = max(detected_ranks)` else 128
- 12B extra args: `--hf-overrides '{"architectures": ["Gemma4ForCausalLM"], "model_type": "gemma4"}'`

The starter notebook uses `gpu_memory_utilization=0.90`, `startup_timeout=1200`, and `VLLM_MEMORY_PROFILER_ESTIMATE_CUDAGRAPHS=1` (S6:20,280,287). The env var is critical for LoRA: without it, vLLM may not budget CUDA-graph memory and can OOM during sampler warm-up (S26). The host says the starter notebook uses the same configuration as the scorer (S26).

4.2 Allowed models

Model	Weight footprint	Status
`gemma-4-31b-it-qat-w4a16-ct`	~16–18 GB total	Allowed; starter default (S5:156)
`gemma-4-12b-it-qat-w4a16-ct`	~6–8 GB	Allowed (S5:157)
`gemma-4-e4b-it-qat-w4a16-ct`	~3–4 GB	Allowed (S5:158)
`gemma-4-e2b-it-qat-w4a16-ct`	~2–3 GB	Allowed (S5:159)

At most one unique base model is allowed; declare it explicitly on every agent (S5:170-173). Provider prefixes (`openai/`, `google/`, `hosted_vllm/`, `custom/`) are stripped before comparison. Declaring multiple base models or an unpermitted model raises `ParticipantVisibleError`.

The model registry also has bf16 aliases for local CLI evaluation only (S5:160,178-185); they are not allowed in Kaggle scoring.

4.3 LoRA adapters

Layout and routing (S5:191-201):

- `adapters/<adapter_name>/adapter_config.json` + `adapter_model.safetensors`.
- Adapter names match `[A-Za-z0-9][A-Za-z0-9._-]*`.
- Reference with `adapter: <adapter_name>` on any `LlmAgent`. vLLM serves adapters per request via `--lora-modules` and the `model` field (S21).
- Different agents may use different adapters, but the base model is fixed.
- `discover_adapters()` registers all adapters with vLLM via `--lora-modules`; `resolve_swegemma_adapter` routes each agent to `openai/<adapter_name>`.
- Size planning: rank 16 ~110–220 MB; rank 32 ~220–450 MB; rank 64 ~450–900 MB; rank 128 ~0.9–1.8 GB (S5:197-201). These are planning ranges for 31B all-linear adapters, not exact. A measured rank-16 all-seven 31B adapter was ~245 MB (122.4M parameters at bf16; S25), so the low end of the range assumes fewer modules or a lower dtype. Use `du -sh` after export for the 3 GiB budget.

The adapter must be compatible with the **exact base revision** used at scoring. A community report found that serving a different revision of the same repo silently cost quality (15/17 → 17/17) (S23). Pin the base revision in your records.

Critical validation: vLLM has had Gemma 4 bugs where adapters load successfully but are ignored (S23). Always compare outputs with and without the adapter at temperature 0. If they are byte-identical, the adapter is not active. Do not trust “Loaded new LoRA adapter” logs.

W4A16 stability: an upstream issue on a different compressed-tensors W4A16 model found rank-32 all-layer adapters weak/non-reproducible, rank-16 reproducible, and rank-8 partial-coverage stable (S24). This is not Gemma-specific, but the competition base is also W4A16. Start at rank 8–16 and test; do not assume rank 64 is safe just because the host has seen rank-64 submissions score (S26).

---

5. Two-phase evaluation lifecycle

Every task runs in two separate sandbox lifecycles (S5:204-230):

1. Start Container A, stage wheels.
2. Extract the repository snapshot at `base_commit` into `/workspace`.
3. Remove future git history; configure `.git/info/exclude`.
4. Editable install and test dependency staging.
5. Write `pytest.ini` and a hermetic `conftest.py`.
6. Commit the baseline.
7. Start the agent timer and send the task prompt.
8. Agent loop: tools, nudges, budgets.
9. Extract `agent_patch` (submitted or fallback).
10. Destroy Container A.
11. Start fresh Container B; repeat baseline setup as `eval_baseline`.
12. Apply `agent_patch` with four fallback passes.
13. Reset protected test/config paths.
14. Apply `test_patch`.
15. Run hermetic pytest with JUnit XML.
16. `exit_code == 0` **and** JUnit XML valid (see §9.2) → resolved.

The agent timer starts **after** container creation, wheel installation, and snapshot extraction (S5:511). Container setup is therefore excluded from the per-task agent time budget, but the global 12h limit still includes setup (S1). This is why local timing can under-estimate the global run.

5.1 Sandbox limits

- Docker backend default: `swebench-sandbox:latest` (S5:234).
- Network: `none`, completely air-gapped (S5:235).
- Memory: 4 GiB; OOM kills with exit code 137 (S5:236).
- CPU: 2 vCPUs (S5:237).
- Working dir `/workspace`, `TEST_TMPDIR=/tmp` (S5:238).
- Containers are not reused by default; warm pooling wipes `/workspace` and `/tmp` (S5:239).
- Subprocess backend: isolated temp dir, rewritten paths, process-group kill on timeout (S5:240-242).

5.2 Bootstrap details that matter

- `pytest.ini` and `conftest.py` are written by the harness **before** the baseline commit, so they are in HEAD. Do not edit or delete them unless the task genuinely requires it; edits appear in your diff (S5:263-268,624).
- The harness appends `__pycache__/`, `*.pyc`, `.pytest_cache/`, `*.egg-info/`, `build/`, `dist/`, `.coverage` to `.git/info/exclude` (S5:250-257). These don't enter the patch, but

scratch `.py` files do.

- Editable install: `pip install --no-index --find-links=/wheels --no-build-isolation --no-deps -e /workspace` (S5:259-260). The repo is importable without network.
- Test dependencies are streamed from cached site-packages; `/sandbox/setup.py --fast-path` handles repo-specific setup (S5:262).

---

6. Harness-to-agent protocol

6.1 Session state

The ADK session is created with `app_name='swegemma_eval'`, `user_id='eval_user'` (S5:273-278). State contains:

- `problem_description` = task problem statement.
- `hints` = stripped `hints_text`, only if non-empty.

`{variable_name}` placeholders in `instruction` are interpolated from session state, so you can use `{problem_description}` and `{hints}` directly (S5:278). The harness also sends them in the user prompt; don't duplicate them unless you have a reason.

6.2 Initial prompt

The harness sends up to seven sections (S5:282-319):

1. Repository and problem statement.
2. Hints, if non-empty.
3. Task budget: time, tool calls, max loop iterations, cost.
4. Execution environment rules: 300 s command timeout, 5,000-char output, 150 lines/10,000 chars per `read_file`, offline notice.
5. Standard instructions: work under `/workspace`, inspect conventions, verify with targeted tests, call `submit_patch`, return a final response.
6. Code intelligence guidance, only if graph files exist and graph tools are declared.
7. Workspace layout: up to 200 BFS entries, excluding hidden/caches and pruning docs/tests/examples below depth 1.

The prompt is filtered against declared tools; undeclared tools are never advertised (S5:282). `enable_sandbox_testing=True` by default, so pytest/unittest are available in Container A; if false, the harness masks them and instructs inline assertions (S5:311).

6.3 Termination and nudges

- `submit_patch()` sets `patch_submitted=True`; the loop breaks as soon as a final text response is emitted or the turn ends with `patch_submitted` (S5:328-329).
- `max_nudges=3`; a turn with at least one valid tool call resets the consecutive nudge count; otherwise it increments and terminates above 3 (S5:330-333).
- Nudges:
  - unclosed `<|tool_call>` tag → split edits, emit the next call immediately;
  - `MAX_TOKENS`/`LENGTH` → keep reasoning short, emit the next call or submit;
  - normal stop without submit → continue or submit (S5:335-338).

Tool errors are returned to the model as structured JSON instead of aborting the task (S5:325). This is helpful, but repeated malformed calls can still loop until the budget ends. The prompt should explicitly say “never repeat an identical failed call.”

---

7. The nine tools

All tools return JSON. Success: `{"status":"ok",...}`. Error: `{"status":"error","error_type":...,"error_message":...,"details":...}` (S5:345-346). When the configured tool-call budget is ≥20 and ≤10 calls remain, every response includes a `budget_warning` (S5:348-350).

Tool	Counts toward calls	Key behavior
`run_command(command)`	Yes	`/bin/bash -c` in `/workspace`; timeout `min(300, max(5, remaining))`; stdout/stderr each truncated to 5,000 chars (S5:353-362)
`submit_patch()`	**No**	`git add -N .`; `git diff --binary _swegemma_baseline` or `HEAD`; sets `patch_submitted`; ends loop after turn (S5:363-370)
`get_status()`	**No**	Live budget/patch status (S5:372-390)
`read_file(filepath, start_line?, end_line?)`	Yes	1-indexed inclusive; cap 150 lines and 10,000 chars; `is_truncated` (S5:396-411)
`edit_file(filepath, old_string, new_string, allow_multiple=False)`	Yes	3-tier matching: exact → flexible whitespace/indent → regex token join; unique match required (S5:413-431)
`write_file(filepath, content)`	Yes	Creates/overwrites; `mkdir -p` (S5:433-439)

Tool	Counts toward calls	Key behavior
`get_code_neighbors(node, edge_type?, max_neighbors=50)`	Yes	Incoming/outgoing graph neighbors; suffix/case-insensitive resolution; 5,000-char cap (S5:445-454)
`search_similar_code(query, k=10)`	Yes	Cosine similarity over node embeddings; query resolved against node keys/suffixes; lexical fallback; 5,000-char cap (S5:456-470)
`get_code_subgraph(nodes)`	Yes	Induced subgraph; 5,000-char cap (S5:472-486)

Path handling: file paths resolve under `/workspace`; leading `/` or `/workspace/` is stripped; `..` raises `ValidationError`; string-encoded ints/bools are coerced (S5:394).

Practical implications:

- `read_file` is capped at 150 lines/10,000 chars; use line ranges.
- `run_command` output is capped at 5,000 chars; use `rg -n`, `sed -n`, `head`, not `cat`.
- `edit_file` requires a non-empty existing file and a unique match; huge edits can truncate the tool call. Use small incremental edits.
- `submit_patch` is free; `get_status` is free. Use them.
- Graph tools only work when graph/embedding files exist; public audits found them unhelpful on many tasks (S36–S38). The host says private tasks have graphs and no async-function dependency (S33). Test before relying on them.

---

8. Budgets and context management

8.1 Defaults

Item	Default	`eval_config.yaml` key
Session wall clock	60.0 min (sample: 1 min)	`evaluation.max_time_minutes`
Max tool calls	100 in inference.py / None in CLI (sample: 10)	`evaluation.max_tool_calls`
Max reasoning turns	500 (sample: 50)	`evaluation.max_turns`
Max cost	None	—
Max total tokens	None	—

Item	Default	`eval_config.yaml` key
Phase-1 command timeout	300 s	`evaluation.timeout_seconds`
Phase-2 pytest timeout	300 s (fixed on scorer)	—
Max command/diff/graph output	5,000 chars	—
Max lines per `read_file`	150	—
Max chars per `read_file`	10,000	—

Source: S5:499-509. Container setup is excluded from the agent budget (S5:511). `cost_usd` and `total_tokens` default to `None`; they are not a practical cap unless you set them.

8.2 Compaction and caching

- `EventsCompactionConfig`: `compaction_interval=15`, `overlap_size=2`, `token_threshold=14,336`, `event_retention_size=5` (S5:518-521).
- `ContextCacheConfig`: `min_tokens=2,048`, `ttl_seconds=1,800`, `cache_intervals=10` (S5:523).
- Retries: `ModelRetryPlugin` handles 429/500/502/503/504 and transient API errors, up to 5 retries with exponential backoff (S5:525).

The context window is 32,768 tokens total. Compaction can drop tool results or instructions; keep the system prompt short and the critical rules few. A community report says compaction may lose tool results because the summary keeps only text parts; the host said they would look into it (S28). Assume compaction can lose information and design accordingly.

8.3 The 12-hour global limit

The harness README does not state a global 12h limit (S5). The competition overview does: 12 hours for all tasks, inclusive of sandbox setup, excluding validation (S1). Participants report that exceeding it produces “Notebook Timeout” with no partial score (S29). The public 0.18/0.22 notebook’s 8-minute per-task cap is not a guarantee of global completion (S37); the starter notebook’s cap is 1 minute (S5:493-497).

Plan with a measured mean, not the cap:

- If mean task time is ~5 min and there are ~120 tasks, agent time is ~10h; setup/startup adds more.
- If mean task time is ~5.5 min, agent time is ~11h; little margin.
- If mean task time is ~8 min, agent time is ~16h; timeout unless most tasks finish early.
- Local timing on a different GPU/topology under-estimates hidden-repo test time (S29).

Use `max_time_minutes` as a per-task safety valve, not a target. Measure p50/p95 and project the global run.

---

9. Patch extraction and Phase 2 verification

9.1 Extraction

Both `submit_patch()` and the fallback run (S5:529-537):

```
cd /workspace && git add -N .
cd /workspace && (git diff --binary _swegemma_baseline 2>/dev/null || git diff --binary
HEAD)
```

`git add -N .` includes new untracked files. `.git/info/exclude` excludes caches/build artifacts. Protected hunks are stripped (S5:535):

- `test_*.py`, `*_test.py`, `.py` under `tests/`, `test/`, `testing/`;
- `conftest.py`, `pytest.ini`, `pyproject.toml`, `tox.ini`, `setup.cfg`, `.pytest.ini`, `sitecustomize.py`, `usercustomize.py`, `_swegemma_stubs.py`, `*.pth`.

If the agent never calls `submit_patch`, the harness still runs the same diff and stripping before tearing down Container A (S5:536). A non-empty diff over non-protected files is recorded and graded (S5:537). So an unsubmitted edit can still score, but explicit submission is safer and free.

Patch hygiene:

- Scratch files in `/workspace` enter the patch; put them in `/tmp` (S5:625).
- Do not edit `pytest.ini`/`conftest.py`; they are in the baseline (S5:624).
- Do not edit tests; those hunks are stripped (S5:535).
- Use `git status --short && git diff` before submitting; `git diff` alone hides untracked files (S36).
- New source files are okay; new test files are stripped.
- Large/binary patches are riskier; keep changes source-only and text.

9.2 Phase 2

A fresh Container B is created (S5:540-567):

1. Original snapshot at `base_commit`, synthetic hardware stubs, `.git/info/exclude`, editable install, cached test deps, `pytest.ini`/`conftest.py`, `eval_baseline` commit.
2. Apply `agent_patch` with four passes:
    - `git apply --unsafe-paths` (`-p1`), then `-3`, then whitespace-ignore, then `--recount`;

    - symlink-normalized `git apply`;
    - prefixless `-p0`;
    - GNU `patch -p1`/`-p0` with dry-run.
3. If all four fail → `resolved=False`, error “Failed to apply agent patch” (S5:548).
4. Reset protected test/config paths referenced by `test_patch` and touched by `agent_patch` to `eval_baseline`; `git clean -f` (S5:549-553).
5. Apply `task.test_patch`.
6. Run:

```
cd /workspace && PYTHONSAFEPATH=1 PYTHONNOUSERSITE=1 python3 -s -m pytest <targets> \
  --junitxml=/tmp/_swegemma_junit_<id>.xml \
  -p no:anyio -o timeout=0 \
  -o python_classes="Test* *Test" -q
```

7. Resolved iff exit code 0 and JUnit XML has `passed_tests > 0`, `failures == 0`, `errors == 0`, and every required node passed and was not skipped (S5:565-567).

The test runner conventions matter: `-p no:anyio`, `PYTHONSAFEPATH=1`, `PYTHONNOUSERSITE=1`, `python_classes="Test* *Test"`. Some real fixes may fail under these conventions; that is the scorer’s behavior, not a bug you can fix. The host says hidden tasks validate 100% with gold patches (S31).

---

10. Local CLI and artifacts

The `swegemma` CLI can evaluate a submission directory against the public tasks (S5:584-620):

```
swegemma eval \
  --tasks tasks.jsonl \
  --snapshots-dir snapshots \
  --submission-dir sample_submission \
  --results-dir results/run_01 \
  --sandbox docker \
  --max-tool-calls 50 \
  --max-time-minutes 30 \
  --concurrency 2 \
  --display auto
```

CLI `--max-tool-calls`/`--max-time-minutes` override the submission’s `eval_config.yaml` values; use one source of truth when measuring a bundle.

Flags:

- `--task-id` / `--task-ids` — specific tasks.
- `--sandbox {docker,subprocess}` — Docker default; subprocess for no-Docker environments.
- `--timeout-seconds` — Phase-1 command timeout.

- `--verification-timeout-seconds` — Phase-2 pytest timeout.
- `--concurrency` — parallel tasks; not representative of the scorer's sequential global time.
- `--shard-index` / `--num-shards` — deterministic sharding.
- `--models-yaml` — custom model aliases/pricing.
- `--skip-agent-patch` — bypass Phase 1 to check baseline test failures.

Results directory (S5:608-620):

```
results/run_01/
├── summary.json
├── task_results.jsonl
├── patches/<instance_id>.patch
├── test_outputs/<instance_id>.log
├── traces/trace_<instance_id>.json
└── logs/<instance_id>.log
```

`traces/trace_<instance_id>.json` is an ATIF-compatible `SessionTrace` with steps, thoughts, tool calls, and token usage (S5:22,617). This is the raw material for post-training data.

Local evaluation caveat: the starter notebook uses `sandbox='subprocess'`, 2 tasks, and sample budgets (S6:351,362). Plain-notebook gold patches fail on many public tasks due to missing test dependencies and wheel shadowing; a calibrated sandbox gets 127/129 (S31, S32). Do not use a plain notebook as your only local evaluator.

---

11. Known harness quirks and ambiguities

These are documented so you don't build on sand:

1. **Global 12h is not in the README.** It is in the competition overview and host/participant reports (S1, S29). Treat it as real.
2. **`_swegemma_baseline` provenance is not documented.** The diff commands reference it, but the bootstrap steps create `baseline`/`eval_baseline` commits. The fallback to `HEAD` usually covers it; don't depend on the ref directly (S5:365,532).
3. **Zip top-level layout is ambiguous.** Use `agent.yaml` at the zip root, as the starter notebook does (S6:600-601).
4. **`max_file_count` scope is ambiguous.** Assume it counts all unpacked files.
5. **`.py` placement is ambiguous.** Keep `.py` under `skills/`.
6. **Phase-2 entry boolean is ambiguous.** The operative rule is "non-empty prediction"; ensure a non-empty patch.
7. **`model` default is ambiguous.** Declare the model explicitly on every agent.
8. **`disallow_transfer_*` semantics are not documented.** Avoid relying on them; use `include_contents: none` and separate agents for isolation.
9. **Cost/token budgets are unset by default.** Don't rely on them to stop a runaway run.
10. **Compaction can lose tool results.** Keep context small; don't rely on long histories (S28).

11. **Graph tools are exact-ID and sometimes unhelpful.** Public audits found them weak (S36–S38); host says private graphs exist and async is not an issue (S33). Test on your tasks.
12. **Tool-call parsing can lose arguments.** Gemma 4 uses special tokens, not JSON; malformed quoting can swallow the next argument (S39). Keep edits small and arguments simple.
13. **LoRA can be silently ignored.** A/B test outputs (S23).
14. **LoRA can reduce KV cache/startup headroom.** Use the env var, one adapter, low rank; test long prompts (S26, S27).

---

12. What to verify before trusting a local run

- Gold patch passes in your environment (or you know which tasks are environment-broken).
- Empty patch fails (or you know which tasks are broken).
- `patch_size > 0` and `files_changed > 0` for a real attempt.
- The patch applies in a fresh container.
- The test output shows the expected tests, not a collection error.
- The adapter changes outputs at temperature 0 (if used).
- The run completes within a projected global budget.
- The trace shows the tool calls you expect, not a loop of identical calls.

If any of these fail, fix the instrument before changing the agent. The public evidence is full of teams that optimized against a broken local evaluator (S31, S32, S39, S40).